

🎤 演讲稿：给 AI 戴上“紧箍咒”——机器人控制系统的 AI 安全驾驶指南

演讲人：子豪

时长：8-10 分钟

核心隐喻：把 AI 比作“超级赛车手”，Harness（管控体系）比作“赛道护栏和交通规则”。

0. 开场白：咱们为什么聚在这里？（1 分钟）

(面向全场，微笑，眼神扫过行政和总工)

各位领导、总工，还有硬件、测试的战友们，大家下午好！

今天我不讲复杂的代码，咱们聊点实在的。现在 AI 写代码很火，咱们团队也在用。但大家心里可能都有个顾虑：**AI 写代码是快，但它懂咱们的机器人吗？**

咱们做的是工业擦窗机器人，不是写网页。咱们的软件要是卡一下，机器人可能就撞墙了；视频画面要是延迟个 1 秒，操作员就瞎了。

以前靠人盯着代码，累得半死。现在 AI 帮忙写，万一它为了“偷懒”写了一段慢代码，或者乱改了硬件参数，那后果不堪设想。

所以，这篇论文解决的核心问题就一个：**怎么让 AI 这个“超级赛车手”，在咱们设定的“安全赛道”里狂飙，既不撞车，又能把活干得漂亮。**我们管这套“赛道规则”叫 **Harness（管控体系）**。

1. 痛点直击：为什么要管 AI？（2 分钟）

(眼神看向硬件工程师和测试人员)

大家想想这几个场景，是不是很熟悉？

- **对硬件和总工来说**：咱们定死的指标，TCP 通信延迟必须小于 **500毫秒**，视频延迟小于 **600毫秒**。这是红线！但 AI 不懂这个，它可能觉得“加个复杂的计算能让代码好看点”，结果导致延迟飙升，机器人直接失控。
- **对测试同事来说**：最怕什么？怕 AI 改了一个小功能，结果把底层的通信逻辑搞乱了。咱们得把整个系统重新测一遍，回归测试的工作量翻倍，心累不累？
- **对行政和领导来说**：咱们这是老项目，70 多个文件的历史代码，推倒重来成本太高。而且如果没有统一规矩，每个人用 AI 写出来的代码五花八门，以后维护起来就是个大坑。

国外大厂做过实验：**同一个 AI 模型，只要给它加上严格的管控规则，代码合格率直接从 6% 飙升到 68%。**

所以，**好用的 AI 开发 = 聪明的模型 + 严格的规矩（Harness）**。这就是我今天要讲的核心。

2. 核心干货：这套“规矩”到底怎么管？（3 分钟）

(手势配合，强调四个动作)

我们没有大动干戈去改机器人硬件，而是在软件层面给 AI 搭了一套“红绿灯系统”，主要干了四件事：

第一，立死规矩（13 本“红宝书”）

我们把所有不能碰的红线，写成了 13 个规则文件。

比如明确规定：“接收机器人数据的线程，必须是最高优先级，雷打不动！”AI 想改代码？行，先读这 13 个文件。不读完，不准动手。

第二，开工前“安检”（5 问自检）

AI 每次要改代码，必须强制回答 5 个问题，就像咱们进车间前的安检：

1. 你会不会增加网络请求拖慢速度？
 2. 你会不会乱改线程优先级？
 3. 你会不会在主线程干耗时操作卡住画面？
 4. 你会不会产生大量垃圾数据？
 5. 你会不会修改关键常量？
- 只要有一个回答是“有风险”，直接驳回，没商量。

第三，规矩打架听谁的？（优先级裁决）

有时候 AI 会纠结，比如“我想简化代码”和“我要保延迟”冲突了怎么办？我们定死了：**安全延迟永远大于代码简洁**。如果两条规矩一样重要，AI 必须举手找人类工程师裁决，绝不让它自己瞎猜。

第四，防止 AI“金鱼记忆”（长效提醒）

AI 聊久了会忘事。我们就把最核心的红线（比如 TCP<500ms）浓缩成一句 **70 个字的“口诀”**，永远贴在它脑门上。每聊 5 轮，就自动让它背一遍口诀，确保它时刻清醒。

落地工具：

我们在代码里加了“**专属标签**”，关键函数上写着“此乃禁区”；同时配了**自动检查脚本**，违规代码根本提交不进仓库，测试同事不用帮 AI 擦屁股。

3. 真实效果：这一个月我们做到了什么？（2 分钟）

（语气自信，展示数据）

这套体系跑了整整一个月（30天），效果非常实打实：

1. **安全零事故**：全程 **0 次** 触碰性能红线，**0 次** 破坏架构。硬件同事最担心的失控风险，被我们在代码阶段就掐灭了。
2. **代码更轻了**：我们把原来 500 多行的通信代码，优化到了 200 行，**砍掉了 58% 的冗余**。代码越少，出错的概率越低，硬件负载也更低。
3. **越用越稳**：刚开始第一周，AI 还会偶尔犯迷糊，一周冲突 6 次；到了第四周，一周只有 1 次。它学会了咱们的规矩，越来越顺手。
4. **老项目也能改**：咱们没推翻老代码，是在 75 个旧文件的基础上渐进式改造的。行政领导不用担心巨额的重构预算，咱们是边开飞机边换引擎。

我们还搞了一套“**五级成熟度打分**”，现在项目已经达到了**第四级（量化管理级）**。这意味着所有的风险都是可视的、可拦截的。

4. 价值与展望：这对大家意味着什么？（1 分钟）

（面向总工和行政）

最后总结一下，这套方案的价值：

- **填补空白**：市面上教 AI 写网页的很多，但教 AI 写**工业实时控制系统**的，咱们这是独一份。
- **降本增效**：硬件少调参，测试少回归，研发少修低级 Bug。大家都轻松。
- **可复制**：这套流程，以后咱们做新项目，或者改造其他老机器人，直接套用就行。

当然，还有点小遗憾。现在主要是靠模拟程序验证，还没法直接连真机全自动测。后续我们会对接硬件测试工具，实现真机自动校验。

一句话总结：这套 Harness 体系，就是给 AI 戴上了**硬件安全的“紧箍咒”**。既让它有孙悟空的本事，又保证它听唐僧的话，绝不乱来。

谢谢大家！

💡 演讲小贴士（针对你的听众）：

1. 眼神交流：

- 讲到“延迟”、“失控”时，看**硬件工程师和总工**，点头示意，这是他们的痛点。
- 讲到“回归测试”、“不用擦屁股”时，看**测试人员**，他们会会心一笑。
- 讲到“成本”、“老项目改造”时，看**行政和领导**。

2. 词汇替换表（遇到这些词自动翻译）：

- **Harness** → 管控体系 / 规矩 / 紧箍咒
- **ADR (架构决策记录)** → 历史经验总结 / 避坑指南
- **CI/Pre-commit** → 自动检查脚本 / 门卫
- **Context Window (上下文窗口)** → AI 的短期记忆
- **棕地项目** → 老项目 / 历史遗留系统

3. 应对提问：

- 如果总工问：“这会不会限制 AI 的创新？”
- 答：“我们在规则里留了‘挑战通道’。如果 AI 觉得规则不合理，它必须提交论证报告，由人来决定改不改规则。安全是底线，创新要在底线之上。”

(AI生成)